

Лабораторная работа №2

Анализ защищённости приложений и программных систем

Разбор: что делали, зачем делали, что закрепить

Части: 2.1 — оценка защищённости разработанного приложения; 2.2 — оценка защищённости сторонних систем

Приложение: `pziis-vault` — веб-реестр с разделением данных на конфиденциальные и неконфиденциальные

Стек: Python 3.12 · FastAPI · PostgreSQL 15 · nginx · Docker

Объём: 2656 строк кода и схемы, 56 автоматических проверок, 12 сценариев демонстрации

1. Место работы в курсе и её структура	2
2. Часть 2.1: что требовалось и что сделано	2
3. Зачем каждое решение: обоснования	4
4. Центральный компромисс: поиск по шифротексту	5
5. Что повторяется, закрепляется и изучается	7
6. Понятия, которые надо знать наизусть	7
7. Вопросы на защите и ответы	9
8. Что показывать на демонстрации	12
9. Остаточные риски: что мы НЕ защитили	13
10. Часть 2.2: что предстоит	15
11. Чек-лист готовности к сдаче	16

1. Место работы в курсе и её структура

Лабораторная работа № 2 — независимая от первой. Внутри неё зависимость есть: часть 2.2 анализирует приложение уровня 2.1.

Методичка 2.2 прямо указывает в разделе «Инструментальные и вспомогательные средства»: «Вариант выполненной лабораторной работы № 2.1». Ссылок на лабораторную работу № 1 ни в 2.1, ни в 2.2 нет. Поэтому четыре отчёта сдаются раздельно, а лаба №1 фигурирует в лабе №2 лишь как переиспользованное окружение и материал для раздела анализа.

Работа	Тема	Артефакт, который сдаётся
1.1	ACL в файловой системе	программа настройки и проверки прав на объекты ФС
1.2	Контроль доступа в СУБД	программы установки, настройки, проверки и удаления СУБД
2.1	Оценка защищённости своего приложения	приложение + анализ его защищённости с обоснованием
2.2	Оценка защищённости сторонней системы	методика оценки + аудит по ней + меры + рекомендации

Смысловая дуга курса читается так: **ЛР 1** — «умею настроить разграничение доступа»; **ЛР 2.1** — «умею построить защищённую систему и обосновать её защищённость»; **ЛР 2.2** — «умею оценить защищённость чужой системы по собственной методике». Каждая ступень меняет роль исполнителя: настройщик → проектировщик → аудитор.

2. Часть 2.1: что требовалось и что сделано

2.1. Требования задания и их закрытие

Задание требует пять функций. Разделение данных на два класса — конфиденциальные и неконфиденциальные — не косметика, а несущая конструкция: вокруг него строится весь дальнейший анализ.

№	Требование	Реализация
1	Добавить пользователя в систему	POST /register (роль <code>guest</code>) и POST /admin/accounts (создание администратором с назначением роли)
2	Авторизовать по идентификационным данным	POST /login : проверка пароля по хешу Argon2id, выдача серверной сессии
3	CRUD и поиск конфиденциальных данных	/secrets/* : тело записей зашифровано AES-256-GCM, поиск через слепой индекс
4	CRUD и поиск неконфиденциальных данных	/notes/* : открытое хранение, полнотекстовый поиск по индексу <code>tsvector</code>
5	Деавторизовать пользователя	POST /logout : удаление строки сессии на сервере и сброс cookie

2.2. Анализ по четырём позициям

Второй пункт задания задаёт четыре ракурса анализа. Это же каркас и в части 2.2, поэтому его стоит запомнить как единый список.

Ракурс	Что реально показывается
Политика безопасности ОС	процесс от непривилегированного <code>pziis</code> , а не от <code>root</code> ; код <code>0750 root:pziis</code> — приложение не владеет своим кодом; ключи <code>0640 root:pziis</code> ; <code>CapabilityBoundingSet=</code> пуст; <code>ProtectSystem=strict</code> ; <code>MemoryDenyWriteExecute=yes</code> ; <code>umask 0027</code>
Компоненты и их взаимодействие	роль <code>vault_app</code> без <code>SUPERUSER</code> , <code>CREATEDB</code> , <code>CREATEROLE</code> , <code>BYPASSRLS</code> и без DDL; на журнал аудита выданы только <code>INSERT</code> и <code>SELECT</code> ; параметризованные запросы; секреты только в окружении; обобщённые сообщения об ошибках
Сетевая политика	TLS 1.2/1.3 только с AEAD-шифронаборами, HSTS; приложение на <code>127.0.0.1</code> ; СУБД на <code>localhost</code> с <code>scram-sha-256</code> ; <code>ufw default deny incoming</code> , открыт только 443; лимит 5 запросов в минуту на <code>/login</code> ; CSP <code>script-src 'none'</code>
Конфиденциальность данных	Argon2id для паролей; AES-256-GCM для конфиденциальных полей; SHA-256 от токена сессии вместо самого токена; разделение ключей через HKDF; администратор без доступа к открытому тексту; никаких конфиденциальных данных в журналах, URL и сообщениях об ошибках

2.3. Архитектура: три доверительные границы

Каждая граница имеет собственный контроль, и это не декорация, а способ изложения материала в отчёте:

1. **Сеть → nginx.** Терминация TLS, ограничение частоты запросов и размера тела, заголовки безопасности. Наружу открыт один порт.
2. **nginx → приложение.** Обмен по петлевому интерфейсу; сервер приложений не слушает внешний интерфейс, поэтому обойти прокси и его лимиты невозможно.
3. **Приложение → СУБД.** Отдельная роль с минимальными привилегиями плюс построчная защита (RLS), привязанная к локальному параметру транзакции `app.actor_id`.

ГЛАВНАЯ МЫСЛЬ, КОТОРУЮ НАДО УМЕТЬ СФОРМУЛИРОВАТЬ

Защищённость не является свойством одного механизма. Она складывается из согласованных решений на четырёх независимых уровнях, а через все уровни проходят три принципа: **минимизация привилегий** (процесс не владеет своим кодом, роль СУБД не умеет DDL, приложение не может стереть журнал аудита), **эшелонирование** (контроль доступа реализован дважды и независимо — в коде и в СУБД) и **называние цены** (безопасность обменивается на функциональность, и обмен нужно проговаривать вслух).

3. Зачем каждое решение: обоснования

На защите спрашивают не «что вы сделали», а «почему именно так». Ниже — обоснование каждого выбора в одну-две фразы.

3.1. Хранение паролей

Решение	Обоснование
Argon2id, а не SHA-256	SHA-256 считается быстро, поэтому современная видеокарта проверяет миллиарды вариантов в секунду. Argon2id требует 64 МиБ памяти на каждую проверку и обесценивает параллелизм GPU и ASIC. Это победитель конкурса Password Hashing Competition.
Соль на каждый пароль	Без соли одинаковые пароли дают одинаковые хеши: применимы радужные таблицы и виден факт совпадения паролей у разных пользователей.
Формат PHC (параметры внутри хеша)	Стоимость можно поднять позже: при следующем успешном входе пароль перехешируется с новыми параметрами (<code>check_needs_rehash</code>).
Минимум 12 символов, а не «обязательный спецсимвол»	Ставка на длину — рекомендация NIST SP 800-63B. Требование спецсимволов заставляет пользователя записывать пароль, не добавляя стойкости.

3.2. Шифрование конфиденциальных данных

Решение	Обоснование
AES-256-GCM, а не AES-CBC	GCM — режим AEAD: даёт конфиденциальность и целостность одним примитивом. CBC не обнаруживает модификацию шифротекста и исторически уязвим к атакам на padding oracle.
Новый nonce на каждую операцию	Повтор nonce на одном ключе полностью разрушает стойкость GCM: раскрывается XOR открытых текстов и компрометируется ключ аутентификации. Поэтому при редактировании запись перешифровывается целиком, а не «доправляется».
AAD = <code>ext_id owner_id key_version field</code>	Связанные данные не шифруются, но аутентифицируются. Это привязывает шифротекст к конкретной записи, владельцу, версии ключа и полю: перестановка шифротекста в другую запись или подмена владельца ломают проверку тега.
HKDF: два подключа из одного мастер-ключа	Один ключ нельзя использовать и для шифрования, и для индексации. Разные метки <code>info</code> дают независимые <code>K_enc</code> и <code>K_idx</code> : компрометация индексации не влечёт компрометацию шифрования.
Колонка <code>key_version</code>	Задел под ротацию ключа без простоя: новые записи пишутся новой версией, старые дочитываются прежней и лениво перешифровываются.

3.3. Сессии и деавторизация

Решение	Обоснование
Ораque-токен, а не JWT	Требование задания №5 — деавторизация. JWT нельзя отозвать до истечения срока подписи, поэтому «выход» с ним был бы фикцией. Удаление строки сессии даёт мгновенный и полный отзыв.

В базе только <code>SHA-256</code> от токена	Дамп таблицы сессий не позволяет переиграть чужую сессию — по хешу токен не восстановить.
Пересоздание идентификатора при входе	Защита от фиксации сессии (CWE-384): токен, навязанный жертве до входа, не станет авторизованным.
<code>HttpOnly</code>	Токен недостижим из JavaScript, поэтому даже успешная XSS-атака не крадёт сессию.
<code>SameSite=Strict</code> + CSRF-токен	Два независимых рубежа против межсайтовой подделки запроса: <code>SameSite</code> не поддерживается совсем старыми браузерами и не покрывает все сценарии.
Двойной срок жизни (30 мин / 12 ч)	Ограничивает окно использования украденного токена и не позволяет сессии жить вечно за счёт активности.

3.4. Контроль доступа

Решение	Обоснование
RLS в СУБД <i>вдобавок</i> к проверкам в коде	Эшелонированная защита в проверяемом виде: ошибка в одном контуре не открывает данные. Даже гипотетически успешная инъекция не выдаст строки чужого владельца.
<code>set_config('app.actor_id', ..., true)</code>	Параметр локален для транзакции, поэтому значение не «протекает» между запросами через пул соединений. Забытый вызов даёт пустую выборку — безопасный отказ по умолчанию.
Владелец таблиц — <code>postgres</code> , не <code>vault_app</code>	Владелец таблицы по умолчанию не подчиняется RLS. Будь владельцем сервисная роль, все политики оказались бы декорацией.
Журнал аудита без <code>UPDATE</code> и <code>DELETE</code>	Скомпрометированное приложение не сможет затереть следы своих действий.
Администратор без доступа к открытому тексту	Осознанное ограничение (CWE-250, разделение обязанностей): компрометация административной учётной записи даёт управление системой, но не раскрывает конфиденциальные данные всех пользователей.

ОТСТУПЛЕНИЕ ОТ МЕТОДИЧКИ, КОТОРОЕ НАДО ПРОГОВОРИТЬ

Методичка 1.2 описывает администратора как субъекта «с полным доступом ко всем подсистемам». В приложении 2.1 это сознательно ограничено: администратор управляет учётными записями, ролями, блокировками, аудитом и неконфиденциальными данными и видит *мета*данные конфиденциальных записей — но не их содержимое. Полнота полномочий сохранена там, где она нужна для эксплуатации. Это стоит согласовать с преподавателем заранее.

4. Центральный компромисс: поиск по шифротексту

Это то место работы, где безопасность прямо обменяна на функциональность. Именно так его и надо подавать — не как «бесплатное» решение.

Противоречие в задании. Требуется «осуществить поиск конфиденциальных данных». Но по шифротексту AES-GCM искать нельзя: он неотличим от случайных байтов. А расшифровывать всю таблицу на каждый запрос — и медленно, и опасно (открытый

текст оказывается в памяти целиком).

Решение — слепой индекс (blind index). При сохранении записи приложение нормализует заголовок (NFKC, нижний регистр, разбиение на токены, отбрасывание токенов короче трёх символов), затем для каждого токена считает

```
tag = HMAC-SHA256(K_idx, owner_id || token)[:16]
```

и складывает теги в отдельную таблицу. Поиск считает тот же HMAC от поисковой фразы и отбирает записи, содержащие все теги фразы; затем записи расшифровываются, и совпадение проверяется по открытому тексту — это отсеивает коллизии усечённого тега.

Что утекает при полном дампе базы

Не открытый текст, но **равенство и частота токенов**: видно, что две записи одного владельца содержат одинаковое слово, и как часто слово встречается. Для словарных полей (например, названий организаций) это делает возможным частотный анализ.

Чем компромисс ограничен — четыре меры

- **Идентификатор владельца входит в вычисление тега**, поэтому одинаковые слова у разных пользователей дают разные теги: анализ не выходит за пределы одной учётной записи.
- **Индексируется только заголовок** — тело записи не индексируется вообще, поэтому частотная статистика не распространяется на весь текст.
- **Тег усечён до 16 байт**: редкие коллизии дают ложные совпадения, которые отбрасываются после расшифровки, и дополнительно размывают статистику.
- **Порог в 3 символа** убирает самые частотные короткие токены.

Отвергнутые альтернативы

Вариант	Почему отвергнут
Детерминированное шифрование поля	Утечка равенства целых значений, а не отдельных токенов — хуже, чем слепой индекс.
Поиск по расшифрованному кэшу в памяти	Открытый текст оказывается в оперативной памяти и может уйти в раздел подкачки.
Полноценное searchable encryption	Несоразмерно объёму лабораторной работы.

5. Что повторяется, закрепляется и изучается

5.1. Повторяется из лабораторной работы №1

Принцип минимальных привилегий и разграничение доступа. В части 2.1 ракурсы «политика безопасности ОС» и «политика безопасности компонентов» — это буквально содержание частей 1.1 и 1.2, но теперь как *один из четырёх* ракурсов анализа, а не как самоцель.

5.2. Закрепляется

Тот же аппарат ACL и ролей, но со сменой роли исполнителя. В лабе № 1 вы **настраивали чужую систему** извне: создавали пользователей, выдавали гранты в PostgreSQL. В 2.1 вы **проектируете свою** так, чтобы она изначально требовала минимума прав, и обязаны это обосновать. Переход «настроить» → «обосновать» и есть закрепление.

5.3. Изучается нового

Тема	В чём новизна относительно ЛР №1
Модель угроз и классификация атак	Перебор, словарная атака, анализ трафика, «человек посередине», инъекции, DoS/DDoS, вредоносное ПО, социальная инженерия. В лабе №1 угроз как таковых не было — только права.
Защита конфиденциальности данных	Хранение (хеширование, шифрование) и передача (TLS) как отдельная дисциплина. Отсюда и Панасенко «Алгоритмы шифрования» в списке литературы — в лабе №1 его нет.
Аутентификация и управление сессиями	Внутри приложения, а не средствами внешней системы.
Сетевая политика безопасности	Совершенно новый ракурс: TLS, периметр, лимиты, заголовки.
Методика оценки защищённости (2.2)	Главное новое умение курса: системный аудит вместо точечной проверки, включая организационные и физические меры.
Анализ стороннего кода и сканирование	Чёрный ящик, сканеры уязвимостей, CWE как классификатор.

6. Понятия, которые надо знать наизусть

Термин	Определение в одну фразу
AEAD	Режим шифрования, дающий конфиденциальность и целостность одновременно (AES-GCM, ChaCha20-Poly1305).
Nonce	Число, используемое один раз с данным ключом; повтор в GCM разрушает стойкость.
AAD	Связанные данные: не шифруются, но аутентифицируются — привязывают шифротекст к контексту.

HKDF	Функция вывода ключей: из одного мастер-ключа получает несколько независимых подключей.
Argon2id	Память-затратная функция хеширования паролей; гибрид Argon2i и Argon2d, победитель PHC.
Соль	Случайное значение на каждый пароль, делающее радужные таблицы неприменимыми.
Слепой индекс	Ключевой хеш (HMAC) от нормализованного токена, позволяющий искать по шифротексту, раскрывая лишь равенство токенов.
RLS	Построчная защита в PostgreSQL: политика, ограничивающая видимые строки на уровне СУБД.
Оpaque-токен	Непрозрачный случайный идентификатор сессии, состояние которого хранится на сервере (в отличие от JWT).
CSRF	Межсайтовая подделка запроса: браузер жертвы выполняет действие на доверенном сайте по инициативе чужого.
XSS	Внедрение сценария в страницу, исполняемого в браузере жертвы с её правами.
CSP	Заголовок, ограничивающий источники ресурсов страницы; главный барьер против последствий XSS.
HSTS	Заголовок, запрещающий браузеру обращаться к сайту по HTTP.
SCRAM-SHA-256	Механизм аутентификации в PostgreSQL, при котором пароль не передаётся по сети.
CWE	Общий перечень слабостей ПО — классификатор типов уязвимостей.
Принцип минимальных привилегий	Субъект получает ровно те права, без которых не может выполнить свою задачу, и ни одним больше.
Эшелонированная защита	Несколько независимых рубежей, где отказ одного не открывает систему.
Триада CIA	Конфиденциальность, целостность, доступность — три свойства, которые защищают.

Перечень CWE, на которые опирается анализ

CWE-89 (SQL-инъекция), CWE-79 (XSS), CWE-352 (CSRF), CWE-384 (фиксация сессии), CWE-522 (недостаточно защищённые учётные данные), CWE-256 и CWE-257 (хранение паролей), CWE-311 и CWE-312 (отсутствие шифрования при хранении), CWE-319 (передача в открытом виде), CWE-250 (излишние привилегии), CWE-209 (утечка через сообщения об ошибке), CWE-798 (защитные учётные данные), CWE-307 (отсутствие ограничения попыток аутентификации), CWE-204 (различие ответов, раскрывающее существование учётной записи), CWE-377 (небезопасный временный файл).

7. Вопросы на защите и ответы

Двадцать вопросов, которые задают чаще всего. Ответ короткий и по существу — без ответа механизм не считается понятным.

В. Почему Argon2id, а не SHA-256 с солью?

О. SHA-256 проектировалась быстрой: видеокарта перебирает миллиарды вариантов в секунду. Argon2id требует 64 МиБ памяти на проверку, что обесценивает параллельный перебор на GPU и ASIC. Соль защищает от радужных таблиц, но не от перебора — от него защищает именно стоимость.

В. Зачем соль, если хеш и так односторонний?

О. Без соли одинаковые пароли дают одинаковые хеши: можно применить заранее посчитанную радужную таблицу и сразу видно, у кого пароли совпадают. Соль делает каждый хеш уникальным.

В. Почему AES-GCM, а не CBC?

О. GCM — режим AEAD: обнаруживает любую модификацию шифротекста. CBC даёт только конфиденциальность, не замечает подмену и исторически уязвим к атакам на padding oracle.

В. Что такое AAD и что было бы без него?

О. Связанные данные аутентифицируются, но не шифруются. Без них шифротекст заголовка можно было бы переставить в другую запись или выдать за запись другого владельца — расшифровка прошла бы успешно. С AAD такая подмена ломает проверку тега.

В. Почему nonce нельзя переиспользовать?

О. В GCM повтор nonce на одном ключе раскрывает XOR открытых текстов и позволяет вычислить ключ аутентификации, то есть подделывать теги. Поэтому при редактировании запись перешифровывается заново с новым nonce.

В. Как вы ищете по зашифрованным данным? Не расшифровываете же всю таблицу?

О. Через слепой индекс: при сохранении для каждого токена заголовок считается `HMAC-SHA256(K_idx, owner_id || token)`, усечённый до 16 байт, и складывается в отдельную таблицу. Поиск считает тот же HMAC от запроса и отбирает записи по совпадению тегов; затем найденные записи расшифровываются и совпадение подтверждается по открытому тексту.

В. Что утекает через слепой индекс?

О. Равенство и частота токенов заголовков в пределах одной учётной записи — для словарных значений возможен частотный анализ. Открытый текст не утекает. Ограничено четырьмя мерами: владелец входит в HMAC, индексируется только заголовок, тег усечён, токены короче трёх символов не индексируются. Это осознанная цена за исполнение требования о поиске.

В. Почему не JWT для сессий?

О. Задание требует деавторизации, а JWT нельзя отозвать до истечения срока подписи. С ораque-токеном выход удаляет строку в базе, и доступ прекращается мгновенно. Это проверяется в демонстрации: возврат прежней cookie после выхода не восстанавливает доступ.

В. Зачем RLS, если права уже проверяются в коде?

О. Это второй, независимый контур. Ошибка в коде или успешная инъекция не выдадут строки чужого владельца, потому что фильтрация выполняется самой СУБД. Если параметр `app.actor_id` не выставлен, выборка пуста — безопасный отказ по умолчанию.

В. Почему администратор не видит чужие секреты? Разве это не противоречит заданию?

О. Задание требует полного доступа ко всем *подсистемам*, и он есть: учётные записи, роли, блокировки, аудит, неконфиденциальные данные, метаданные конфиденциальных записей. Ограничен доступ к *содержимому* чужих секретов, и это применение CWE-250: компрометация одной учётной записи не должна раскрывать данные всех пользователей.

В. А технически администратор действительно не может их прочитать?

О. Нет, честный ответ таков: ограничение держится на коде, политике RLS и представлении метаданных, но **не на криптографии** — ключ `K_enc` принадлежит сервису, поэтому сервис технически способен расшифровать любую запись. Криптографическая гарантия требует ключа на пользователя, обёрнутого ключом из его пароля; это первая мера в списке усовершенствований.

В. Как защищаетесь от SQL-инъекций? Почему не фильтруете кавычки?

О. Все запросы параметризованы: значения передаются отдельно от текста запроса на уровне протокола расширенных запросов, поэтому не могут стать частью SQL. Фильтрация кавычек — ненадёжный подход, её обходят. Дополнительно роль приложения не умеет DDL, а RLS ограничивает выборку.

В. Как защищаетесь от XSS?

О. Три рубежа: автоэкранирование Jinja2 по умолчанию, политика CSP со `script-src 'none'` (в приложении нет JavaScript вовсе) и флаг `HttpOnly` на cookie сессии — даже успешный сценарий не украдёт токен.

В. Что такое CSRF и почему `SameSite` недостаточно?

О. Атакующий вынуждает браузер жертвы отправить запрос на наш сайт, где cookie подставится автоматически. `SameSite=Strict` — первый рубеж, но он не поддерживается совсем старыми браузерами. Поэтому каждый изменяющий запрос дополнительно требует CSRF-токена, связанного с сессией.

В. Почему при неверном логине и неверном пароле одинаковый текст ошибки?

О. Разные ответы превращают форму входа в средство перечисления учётных записей (CWE-204). По той же причине при неизвестном логине всё равно считается хеш по фиктивному значению: иначе разница во времени ответа выдавала бы существующие логины.

В. Почему лимит на `/login` стоит в `nginx`, а не в приложении?

О. Argon2 с памятью 64 МиБ сам является ресурсом: поток запросов на вход исчерпал бы оперативную память. Поэтому лимит стоит *до* приложения. В приложении есть свой лимит как страховка на случай запуска без прокси.

В. Тогда зачем ещё и блокировка учётной записи?

О. Лимит на адрес (5 в минуту) связывает раньше порога блокировки (10 неудач), поэтому с одного источника блокировка недостижима. Она адресована *распределённому* перебору по одной учётной записи, когда лимит на адрес не помогает. Это три разных рубежа против трёх разных сценариев.

В. Что произойдёт при утечке дампа базы?

О. Пароли не восстанавливаются (Argon2id с солью), токены сессий не переигрываются (в базе только их SHA-256), конфиденциальные записи не читаются (ключ лежит в файловой системе, не в базе), подмена записи обнаруживается (AEAD с AAD). Утекает частотная статистика токенов заголовков в пределах учётной записи и метаданные: количество записей, размеры, время изменения.

В. Что произойдёт при компрометации `root` ?

О. Раскрывается всё: `root` читает файл с ключом и память процесса. Защита от вредоносного администратора ОС средствами приложения невозможна и не заявляется. Шифрование защищает от кражи дампа, резервной копии или диска — не от полного захвата машины.

В. Что вы НЕ защитили и почему?

О. Компрометацию `root`, распределённый отказ в обслуживании (нужен вышестоящий фильтр или CDN), социальную инженерию (нужны организационные меры), уязвимости в самих зависимостях (нужен регулярный `pip-audit`). Сертификат на стенде самоподписанный, поэтому защита от «человека посередине» формально неполна до выпуска сертификата доверенным удостоверяющим центром.

8. Что показывать на демонстрации

Методичка требует не только демонстрации работы, но и демонстрации и обоснования защищённости. Обычно это самое слабое место на сдаче.

№	Сценарий	Что видно
1	Хранение паролей	в выдаче <code>SELECT login, pwd_hash</code> — строки Argon2id PHC, ни одного открытого пароля
2	Шифрование записей	<code>SELECT * FROM app.secret_record</code> от имени <code>postgres</code> — только шифротекст; рядом тот же список в интерфейсе владельца в открытом виде
3	Работа RLS	пользователь В не видит запись пользователя А ни через интерфейс, ни напрямую в <code>psql</code> с чужим <code>app.actor_id</code> ; без параметра — ноль строк
4	Ограничение администратора	метаданные видны, открытый текст — нет
5	Поиск по шифротексту	запись находится по слову из заголовка, а в таблице индекса лежат только 16-байтовые HMAC-теги
6	SQL-инъекция	<code>' OR 1=1 --</code> ищется как обычный текст; таблица на месте
7	XSS	<code><script></code> выводится как текст; в заголовках виден CSP
8	Деавторизация	возврат прежней cookie после выхода не даёт доступа; контрпример: с JWT токен остался бы валидным
9	Перебор	три рубежа отдельно: лимит <code>nginx</code> , лимит приложения, блокировка учётной записи; в журнале аудита — события <code>login/denied</code>
10	Привилегии ОС	процесс от <code>pziis</code> , <code>CapEff: 0000000000000000</code> , запись в каталог с кодом — отказ
11	Сетевая политика	наружу опубликован только порт <code>nginx</code> ; TLS 1.3 и <code>TLS_AES_256_GCM_SHA384</code>
12	Целостность журнала	<code>DELETE FROM app.audit_event</code> под ролью приложения — <code>permission denied</code> ; DDL — тоже отказ

ВСЁ ЭТО ИСПОЛНЯЕМО

Сценарии реализованы двумя скриптами. `./check.sh` — 56 автоматических проверок (17 криптографических без базы и 39 сквозных). `./demo.sh` — те же 12 сценариев с выводом реальных строк из базы. Проверки запускаются в том же образе и от того же непривилегированного пользователя, что и приложение: иначе проверка привилегий не имела бы смысла.

СИЛЬНЫЙ АРГУМЕНТ: ПРОВЕРКИ НАШЛИ НАСТОЯЩИЙ ДЕФЕКТ

Первый прогон упал на проверке CSRF: `hmac.compare_digest` отказывается сравнивать строки с символами вне ASCII, поэтому присланный атакующим токен с кириллицей давал ответ **500 вместо 403** — то есть необработанное исключение прямо на пути проверки защиты, на данных, которые атакующий контролирует целиком. Исправлено сравнением на байтах.

Это ровно тот случай, ради которого проверки и пишутся: механизм был спроектирован верно, а его реализация давала сбой. На защите это лучший аргумент в пользу того, что анализ был настоящим, а не декларативным.

9. Остаточные риски: что мы НЕ защитили

Раздел, который отличает анализ от рекламы. Формулировка «всё защищено» на защите не принимается.

Риск	Почему остаётся и чем ограничен
Частотный анализ слепого индекса	Проектная цена за требование поиска. Ограничен пределами одной учётной записи, только заголовками, усечением тега.
Ключ и шифротекст на одном хосте	Компрометация хоста раскрывает данные. Шифрование защищает от кражи дампа, копии или диска — не от захвата машины. Решение: внешнее хранилище ключей.
Сервис способен расшифровать любую запись	Ограничение полномочий администратора логическое, не криптографическое. Решение: ключ данных на пользователя.
Самоподписанный сертификат	Защита от «человека посередине» неполна. Решение: сертификат доверенного удостоверяющего центра.
Открытый текст в памяти процесса	Python не позволяет надёжно затирать строки; возможна утечка через раздел подкачки. Решение: <code>swapoff</code> или шифрование swap.
Распределённый отказ в обслуживании	Мерами одного хоста не отражается. Реализованные лимиты рассчитаны на одиночный источник.
Уязвимость в зависимости	Анализом не покрывается. Компенсация: фиксация версий и регулярный <code>pip-audit</code> .
Метаданные не скрыты	Количество записей, размеры и время изменения видны при доступе к базе.
Компрометация <code>root</code>	Средствами приложения не закрывается и не заявляется.
Социальная инженерия	Требует организационных мер и инструктажа.

Комплекс мер по усовершенствованию — первые две снимают главные риски

- Ключ шифрования на пользователя.** Случайный ключ данных на учётную запись, обернутый ключом, выведенным из пароля через Argon2id. Делает ограничение «администратор не читает секреты» криптографическим. Цена: смена пароля требует перешифровки обёртки, потеря пароля означает потерю данных.
- Сертификат доверенного удостоверяющего центра** вместо самоподписанного — закрывает единственный оставшийся разрыв в защите канала.

Далее: внешнее хранилище ключей (HSM или KMS), общий счётчик лимитов вместо памяти процесса, `pip-audit` в регулярном прогоне, шифрование раздела подкачки, двухфакторная аутентификация для администратора, вынос журнала аудита на отдельный носитель, ротация ключа через заложенную колонку `key_version` .

10. Часть 2.2: что предстоит

Главный артефакт части 2.2 — не разбор одного приложения, а воспроизводимая методика.

10.1. Что требуется сдать

1. **Разработать методику** оценки защищённости систем.
2. **Применить методику** к предоставленному приложению по тем же четырём ракурсам, что в 2.1.
3. **Предложить комплекс мер** по усовершенствованию.
4. **Предложить рекомендации** по безопасному использованию.
5. Оформить отчёт из семи разделов.

10.2. Что методика обязана покрывать

Методичка перечисляет аспекты прямо, и их стоит взять как оглавление методики:

1. Физические устройства хранения и обработки данных.
2. Операционная система и программное окружение.
3. Исходный код на предмет потенциальных уязвимостей.
4. Компоненты системы: СУБД, библиотеки, веб-серверы, протоколы.
5. Взаимодействие между компонентами.
6. Организационные меры пользования системой.
7. Пользование системой на предмет неожиданных результатов, нарушающих целостность, доступность или конфиденциальность.

10.3. Что уже готово от части 2.1

Требование 2.2	Готовый материал
Четыре ракурса анализа	Раздел 2.2 этого разбора — тот же каркас
Матрица «атака → мера → остаточный риск»	17 атак с привязкой к 14 CWE — готовый инструмент методики
Комплекс мер по усовершенствованию	9 мер, упорядоченных по отношению «снятый риск / трудоёмкость»
Рекомендации по безопасному использованию	7 пунктов: хранение ключей, резервные копии, разделение ролей, блокировка вместо удаления, просмотр журнала аудита

ОРГАНИЗАЦИОННЫЙ ВОПРОС, КОТОРЫЙ НАДО РЕШИТЬ ЗАРАНЕЕ

Что именно анализировать в 2.2 — приложение одногруппника или вариант, выданный преподавателем? От этого зависит, будут ли исходники, а значит — доступен ли анализ «белого ящика» или только чёрный ящик со сканерами. Список инструментов OWASP в методичке дан именно для второго случая.

Приложение из 2.1 сознательно спроектировано так, чтобы его же можно было отдать на аудит: слепой индекс, ключ на одном хосте с шифротекстом и самоподписанный сертификат — реальные находки для чужой методики.

11. Чек-лист готовности к сдаче

Часть 2.1

- Приложение поднимается одной командой и работает по HTTPS.
- Все пять требуемых функций демонстрируются вживую.
- Данные разделены на два класса на уровне хранения, а не «по договорённости».
- Анализ написан по четырём ракурсам, в каждом — раздел остаточных рисков.
- Для каждого утверждения о защищённости есть способ его проверить.
- Готов ответ на вопрос «что вы НЕ защитили».
- Отступление про полномочия администратора согласовано с преподавателем.
- Отчёт оформлен по шаблону БГУИР; в выводе есть семантическая и прагматическая составляющие.

Часть 2.2

- Выяснено, какую систему анализировать и есть ли исходный код.
- Методика написана как процедура, применимая к любой системе, а не как разбор одной.
- Методика покрывает все семь аспектов из методички, включая физические и организационные меры.
- Анализ выполнен *по* методике, и это видно по структуре отчёта.
- Меры усовершенствования и рекомендации по использованию — отдельными разделами.

ВЫВОД, КОТОРЫЙ СТОИТ ДЕРЖАТЬ В ГОЛОВЕ ЦЕЛИКОМ

Семантика. Защищённость приложения не является свойством одного механизма: она складывается из согласованных решений на четырёх независимых уровнях — прав в операционной системе, привилегий и политик в СУБД, конфигурации сетевого периметра и криптографической обработки самих данных. Через все уровни проходит минимизация привилегий, контроль доступа эшелонирован в два независимых контура, а безопасность обменивается на функциональность — и обмен нужно называть вслух.

Прагматика. Получен воспроизводимый и проверяемый результат: схема данных с разделением классов информации, ролевая модель, набор конкретных конфигураций и матрица «атака → мера → остаточный риск», привязанная к CWE. Каждое утверждение о защищённости сопровождается способом проверки, что превращает раздел анализа из декларации в исполняемый чек-лист. Одновременно зафиксированы границы применимости: защита не распространяется на компрометацию `root`, на распределённый отказ в обслуживании и на социальную инженерию — эти риски требуют мер, лежащих вне приложения.